

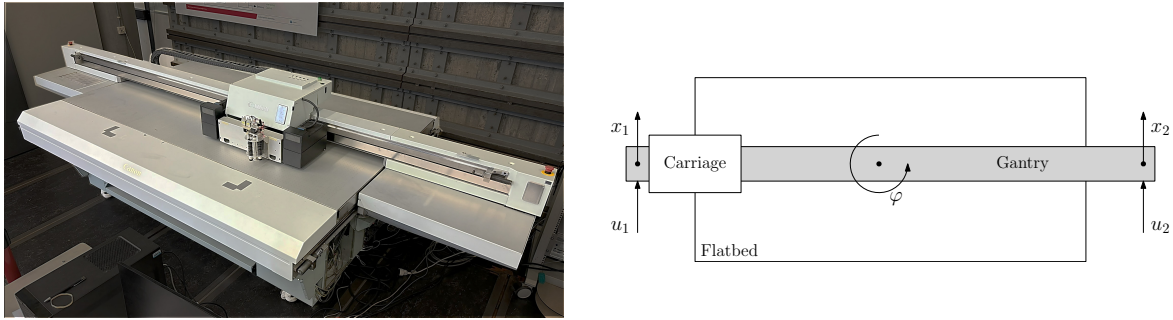


Figure 1.1: Arizona flatbed printer.

1 Introduction

Learning control is a key research topic within the Control Systems Technology Section, which currently is an active research area (see, e.g., [1, 2]). The aim of this course is to address open challenges in the field. In the first weeks, you followed a tutorial on learning control, made exercises, and implemented several of your own Iterative Learning Control (ILC) and Repetitive Control (RC) controllers in experiments with an A3 printer. During the rest of the course, you are going to develop a multi-input multi-output (MIMO) feedforward controller and input shaper and validate the design on the Arizona flatbed printer shown in Figure 1.1.

Many mechatronic systems perform repeated tasks, such as printing, pick-and-place, and wafer processing. In these systems, the same reference is applied repeatedly in a batch-to-batch manner. At the start of a new batch, the reference is changed resulting in a performance decrease when not taken into account in the learning controller design. When printing with the Arizona, the material to be printed is fixed to the flatbed and the gantry together with the carriage move over the to-be printed material. This sequence is as follows: the carriage performs one swath (forward + backward motion) and returns to its starting position on the left, then the gantry performs a step in x-direction, and the carriage performs the next swath. This is repeated until the whole material is printed. The step reference applied to the gantry may change from batch-to-batch, and therefore requires careful design of the learning controller to enable high performance for all references.

Iterative learning control with polynomial basis functions is known to be effective in dealing with batch-to-batch variations, and has been successfully applied in industrial applications, see, e.g., [3] and [4, 5] for MIMO ILC with basis functions. However, ILC with polynomial basis functions tacitly assumes that the system at hand has a unit numerator and is thus unable to fully capture the dynamics of the system, which limits performance.

For rational systems, ILC with basis functions can be extended by introducing an input shaper next to the feedforward controller. Combining a feedforward controller with an input shaper enables to perfectly capture the dynamics of a rational system, and thus high performance can be obtained for changing references. Combining a feedforward controller with an input shaper is initially investigated in [6], where promising results are reported for a single-input single-output (SISO) system.

Although promising results on feedforward control with input shaper have been reported for SISO systems, it remains unclear how to develop such a controller for MIMO systems, how to design the input shaper and feedforward controller, and to learn both in an ILC setting.

The aim of this research challenge is to iteratively learn a MIMO feedforward controller with input shaper for the gantry of the Arizona, and to validate the design on the Arizona.

2 Problem description

Consider the control configuration shown in Figure 2.1a. The multivariable two-input two-output system is given by P . The system is partially decoupled via static input and output transformations, T_u and T_y , respectively, so that C has been designed as a diagonal stabilizing feedback controller. The decoupled system $\tilde{P}$ is given by

$$\tilde{P} = T_y P T_u,$$

so that the transformed input and output are described by $\begin{bmatrix} x \\ \varphi \end{bmatrix} = \tilde{P} \begin{bmatrix} u_x \\ u_\varphi \end{bmatrix}$. The system and controller are considered to be discrete time, multi-input multi-output, and linear time-invariant (LTI) systems. The input shaper is denoted by C_j^y and the feedforward controller by C_j^{ff} . Furthermore, let r_j be the reference signals, r_j^y the shaped references, e_j^y the loop error signals, f_j the feedforward commands, y_j the system outputs, and j the trial index.

Let the system be in the configuration shown in the right plot of Figure 1.1, i.e., carriage at the left position. The step reference that is typically applied to the x -direction of the gantry is shown in Figure 2.1b, whereas the φ reference is zero. For fast point-to-point motion tasks, a settling error $e_j = r_j - y_j$ is present during the dwell time, which is the time between T_1 and T_2 in Figure 2.1b. This settling error causes inaccuracies in both directions x and φ when printing, and thus must be minimized.

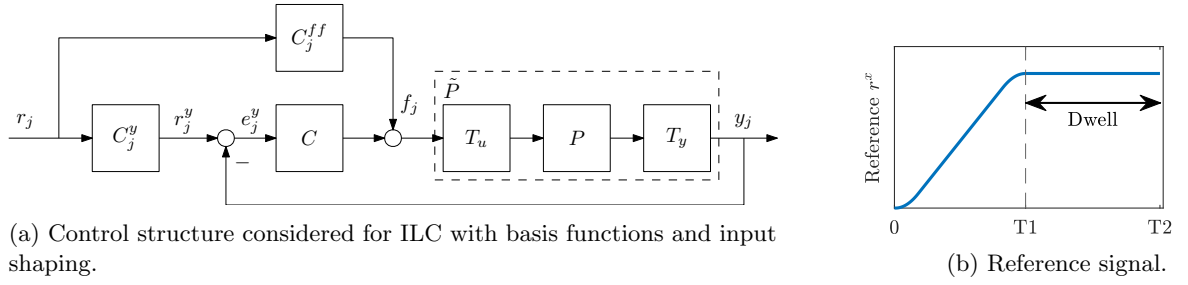


Figure 2.1

The goal is to minimize the settling errors, i.e., the errors during the dwell time, and can be obtained by minimizing the loop error $e_j^y = \begin{bmatrix} e_j^x \\ e_j^\varphi \end{bmatrix}$ (see Remark 2.1), by iteratively learning a multivariable feedforward controller C_j^{ff} and input shaper C_j^y . The resulting controllers should fulfill the following requirements:

- The settling errors e_j^x and e_j^φ must be minimized.
- The controllers must be robust to reference changes, i.e., the controllers capture the inverse plant dynamics.
- The controllers must be able to deal with the nonlinear effects in the system, such as friction.

Remark 2.1. Let the order of the input shaper C_j^y be denoted by n_a , and assume that $C_j^y(z)|_{z=1} = 1$, then the shaped reference r_j^y is equal to the original reference r_j after n_a samples. Then, the tracking error $e_j(k)$ equals the loop error $e_j^y(k)$ after n_a samples, i.e., $e_j(k) = e_j^y(k)$ for $k \in [T_1 + n_a, T_2]$. Hence, minimizing e_j^y also minimizes the settling error e_j , see [6] for more details.

3 Research directions

For this year's challenge we have created an overview on how you can build up the design of the feedforward controller and input shaper, starting from most basic that probably lead to limiting performance to more advanced techniques that are expected to lead to high performance. The overview is shown in Table 3.1. The first column represents the dimensions of feedforward and/or input shaper, the first row represents the type of controller.

Table 3.1: Research directions and possible combinations to tackle the challenge. The first column represents the dimensions of feedforward and/or input shaper, the first row represents the type of controller.

	C^{ff}	C^{ff} and $C_j^y = z^{-d}$	C^{ff} and $C_j^y = 1 + \sum_{i=1}^{n_a} \left(\frac{1-z^{-1}}{T_s} \right)^i \theta^i$
SISO			
MIMO			

Initially, you could start by designing only a SISO feedforward controller C^{ff} (so no input shaper), and then extend this to a MIMO feedforward controller. A different approach is to introduce the input shaper next to a feedforward controller, starting by only compensating the delay in the system, i.e., by selecting samples delay d in z^{-d} , and then extend this to a more general input shaper, e.g., by selecting the parameters of a finite impulse response (FIR) input shaper, where a derivative basis is used, i.e., $(1 - z^{-1})/T_s$. Of course, you are free to decide how you want to tackle this problem and we encourage you to come with ideas beyond the ones mentioned here.

We have defined some general open questions that require careful investigation in order to achieve high performance controllers. Of course, you are free to come up with your own ideas as well! The questions are as follows

- **MIMO ILC with basis functions and input shaper:** Background information on SISO ILC with basis functions and input shaper is provided in Section 4, but how to extend this to MIMO systems? First start by understanding [6] for a general background on feedforward control with input shaper for SISO systems. Then make sure you understand Section 4 how to learn the controllers in an ILC framework. For extensions to MIMO systems, see [4, 5, 7] for inspiration.
- **Parameterize feedforward and input shaper:** How to parameterize the feedforward controller and input shaper, i.e., how to choose the structure and basis functions inside these controllers? Analyse the MIMO system at hand, the Arizona, to obtain insights in the dynamics of the system, and use these insights to select the structure and basis functions for the feedforward controller and input shaper.
- **Nonlinear compensation:** The setup exhibits nonlinear effects due to for example friction, how to compensate for these effects? See for example [4] for example basis functions.
- **Robustness:** How to ensure robustness of the ILC framework and robustness to changing references?
- **Convergence:** Can you provide theoretical guarantees on the convergence of the learning algorithm? Can you provide insights in the convergence speed?

4 Background on ILC with basis functions and input shaper for SISO systems

Consider Figure 2.1a. Let the input shaper and feedforward controller be defined as follows

$$\begin{aligned} C_{j+1}^y &= C_j^y + C_\Delta^y = 1 + \sum_{i=1}^{n_a} \Psi^i \theta_{j+1}^i, \\ C_{j+1}^{ff} &= C_j^{ff} + C_\Delta^{ff} = \sum_{i=n_a+1}^{n_a+n_b} \Psi^i \theta_{j+1}^i, \end{aligned} \quad (4.1)$$

where $\theta_{j+1}^i = \theta_j^i + \theta_\Delta^i$ are the parameters of the input shaper and feedforward controller, and Ψ^i the associated basis functions.

The tracking error according to Figure 2.1a is given by

$$e_j^y = S \left(C_j^y - PC_j^{ff} \right) r. \quad (4.2)$$

Similarly to (4.2), the error at trial $j + 1$ is given by

$$e_{j+1}^y = S \left(C_{j+1}^y - PC_{j+1}^{ff} \right) r. \quad (4.3)$$

Substituting equation (4.1) into (4.3) results in the predicted error at trial $j + 1$ and is given by

$$\hat{e}_{j+1}^y = e_j^y + S \left(C_\Delta^y - PC_\Delta^{ff} \right) r. \quad (4.4)$$

The objective is to minimize the settling error for the next trial, $\hat{e}_{j+1}^y$ in (4.4), by learning the parameters θ_{j+1} based on measured data of the previous trial for a given set of basis functions Ψ . The optimization problem is defined as

$$\begin{aligned} \mathcal{J}(\theta_\Delta) &= \frac{1}{2} \|\hat{e}_{j+1}^y(\theta_\Delta)\|_{W_e}^2 + \\ &\quad \frac{1}{2} \|\Psi^y(\theta_\Delta^y + \theta_j^y)\|_{W_{r_y}}^2 + \frac{1}{2} \|\Psi^y(\theta_\Delta^y)\|_{W_{\Delta r_y}}^2 + \\ &\quad \frac{1}{2} \|\Psi^{ff}(\theta_\Delta^{ff} + \theta_j^{ff})\|_{W_f}^2 + \frac{1}{2} \|\Psi^{ff}(\theta_\Delta^{ff})\|_{W_{\Delta f}}^2. \end{aligned} \quad (4.5)$$

where $W_e, W_{r_y}, W_{\Delta r_y}, W_f, W_{\Delta f} \in \mathbb{R}^{N \times N}$ are symmetric positive (semi-)definite weighting matrices weighting the error, input shaper, and feedforward controller, respectively.

The objective in (4.5) can be rewritten into least squares form, i.e., $\frac{1}{2} \|Y - X\theta_\Delta\|_2^2$ as

$$\mathcal{J}(\theta_\Delta) = \frac{1}{2} \left\| \begin{bmatrix} W_e^{\frac{1}{2}} e_j^y \\ W_{r_y}^{\frac{1}{2}} r_j^y \\ 0 \\ W_f^{\frac{1}{2}} f_j \\ 0 \end{bmatrix} - \begin{bmatrix} -W_e^{\frac{1}{2}} S \Psi^y & W_e^{\frac{1}{2}} S P \Psi^{ff} \\ -W_{r_y}^{\frac{1}{2}} \Psi^y & 0 \\ -W_{\Delta r_y}^{\frac{1}{2}} \Psi^y & 0 \\ 0 & -W_f^{\frac{1}{2}} \Psi^{ff} \\ 0 & -W_{\Delta f}^{\frac{1}{2}} \Psi^{ff} \end{bmatrix} \theta_\Delta \right\|_2^2, \quad (4.6)$$

where the solution is given by $\theta_\Delta = (X^\top X)^{-1} X^\top Y$, which is equivalent to the solution given in (4.5). The parameters for the next trial are then given by

$$\theta_{j+1} = \theta_j + \theta_\Delta. \quad (4.7)$$

5 Practical details

5.1 Material

You will receive the following:

1. A Matlab script that simulates a SISO ILC loop with basis functions and input shaper.
2. Models of the Arizona, both parameteric and non-parametric.
3. Files needed to run experiments on the Arizona.
4. The reference applied to the gantry.
5. A list of useful sources, see the end of this document.

5.2 Experiments

We recommend that you use Git for version control for this course, but this is up to you; just make sure you think about version control. For the experiments in this course, we will use the Arizona flatbed printer in our new motion lab in Gemini-North, floor 0. When you want to do experiments on the Arizona, you can reserve a time slot in the same [online Excel sheet](#) that was used for the A3 printers but now in the Arizona tab. During the experiments, a TA needs to be present, preferably your own TA, so please discuss the planning with them. There will also be a lab TA available during the testing weeks. The files needed for the experiments, along with an explanation of the experimental setup, will be provided in due time.

5.3 Planning

The current planning for this part of the course is as follows

- Week 3: Lecture on background material for the challenge and first TA session.
- Week 4: Get acquainted with the concept, go through the references provided or even beyond, brainstorm about possible directions for the challenge, and make sure you understand the provided Matlab simulation example.
- Week 5: Guest lecture by Sjirk Koekebakker about Learning control in industry.
- Week 6: Explain you initial ideas to your peers using a 10 minute presentation, followed by a short discussion with all groups (see the canvas schedule for details on the date). If you want to start experiments in week 8, work on an experimental plan.
- Week 7: From this week on you can plan experiments on the Arizona with your team.
- Week 10: Final experiments, test your work. Make sure to record some of your work for the video!
- Week 11: Deadline report & movie

We recommend that you meet regularly with your group. The teaching assistants will meet with their groups at least once a week, and may be available to (occasionally) answer questions on Teams outside meetings.

5.4 Deliverables

The following deliverables are expected. Please see the Canvas page for deadlines for each deliverable.

- A 10-minute midterm presentation in which you present your ideas to receive feedback. Please refer to the planning on Canvas. This is informal, and intended for discussion and alignment. It is a challenge, there is not one right answer!

- The multivariable feedforward and input shaper design procedure, and the Matlab and Simulink files to reconstruct the controllers and run the experiments. Please make sure to document your code and files well, so that others can understand and use them.
- A report on your approach, using the distributed template. This report should include at least a theoretical framework, an explanation of the design and implementation, and an evaluation of the experimental results. Please make sure to use a scientific approach, i.e., make sure that you reference appropriately and ensure that your results are fully reproducible: describe your approach in detail.
- A short (± 5 min.) movie, explaining your approach. We discourage pre-recorded PowerPoint presentations. Other than that, the format is free, use your creativity!

References

- [1] Tom Oomen, “Learning in Machines,” *Mikroniek*, 2018.
- [2] T. Oomen, “Learning for Advanced Motion Control,” *2020 IEEE 16th International Workshop on Advanced Motion Control (AMC)*, 2020.
- [3] J. Bolder, T. Oomen, S. Koekebakker, and M. Steinbuch, “Using iterative learning control with basis functions to compensate medium deformation in a wide-format inkjet printer,” 2014.
- [4] H. J. van Deursen, “Multivariable Iterative Learning Control with basis functions : performance enhancement of an industrial flatbed printer,” tech. rep., Eindhoven University of Technology, 2015.
- [5] W. Eijssermans, “MIMO ILC with basis functions applied to a flatbed printing system,” tech. rep., Eindhoven University of Technology, 2014.
- [6] F. Boeren, D. Bruijnen, N. Van Dijk, and T. Oomen, “Joint input shaping and feedforward for point-to-point motion: Automated tuning for an industrial nanopositioning system,” *Mechatronics*, vol. 24, no. 6, pp. 572–581, 2014.
- [7] L. Blanken, J. Van Zundert, R. De Rozario, N. Strijbosch, and T. Oomen, “Multivariable iterative learning control: analysis and designs for engineering applications,” *Data-Driven Modeling, Filtering and Control*, vol. 123, no. 2019, pp. 1–14, 2019.